EduPlay Project Documentation Report

Technical overview, implementation evidence, and submission placeholders

PROJECT REPORT

Project	EduPlay Smart Educational Media Player
Release baseline	0.1.15 | Source commit add199b
Team	[INSERT TEAM NAME]
Institution and course	[INSERT INSTITUTION AND COURSE]
Supervisor	[INSERT NAME, IF REQUIRED]
Prepared	31 October 2026

Replace all bracketed fields before submission.

Executive Summary

EduPlay is an Electron desktop learning-media player combining lecture playback, speech-to-text captions, transcript search, and local AI assistance. The source also provides transcript-grounded flashcards and scene-based visual storyboards. This report maps these features to the implementation and leaves spaces for the team's actual screenshots, paper details, dataset link, and demo evidence.

The AI features use a loopback Express service and locally installed Ollama models. Whisper generates captions through Python, while FFmpeg decodes media and extracts representative frames. The repository contains no dataset, research paper, or demo recording; those materials must be supplied before submission.

Project at a Glance

Item	Project information
Project type	Cross-platform Electron application for learning from lecture media
Primary users	Students and instructors studying video or audio lectures
Source baseline	EduPlay 0.1.15, commit add199b
Core stack	Electron, JavaScript, Express, Python, Whisper, PyTorch, FFmpeg, Ollama
Local text model	phi3, configurable
Local vision model	qwen2.5vl:3b, configurable
Dataset in repo	None; add approved dataset link in the procedure guide
Paper in repo	None; add exact team paper citation before submission

Problem and Objectives

Lecture recordings can be difficult to review efficiently. Students may replay long sections to locate an explanation, search spoken content, or recall key concepts. EduPlay brings these tasks into one desktop workflow.

- Play common local media with standard controls, speed adjustment, and subtitle tracks.
- Generate timestamped speech captions with Whisper and save an SRT file beside the media.
- Search caption text and jump to matching moments.
- Ask transcript-based questions through a local Ollama text model.
- Generate flashcards with transcript references and timestamps.
- Detect scene changes and explain selected video frames with a local vision model.

SDLC and Development Evolution

EduPlay was developed incrementally through the software development life cycle (SDLC): identify a learning problem, define requirements, choose a familiar and suitable stack, design the data flow, implement features in stages, then test and refine them. The sequence below describes how the project grew from a desktop media player into a local AI-assisted study tool.

SDLC stage	Decision or implementation in EduPlay
1. Problem identification	Long lecture recordings take time to review; learners need faster ways to find, understand, and revisit important material.
2. Requirements	Build a desktop app for local media playback, timestamped captions, transcript search, video question answering, flashcards, and visual storyboards.
3. Technology selection and design	Choose Electron because the team was familiar with HTML, CSS, and JavaScript and wanted a desktop app. Use a local Express service, Python Whisper/PyTorch for transcription, FFmpeg for media/frame work, and Ollama for local model inference.
4. Player implementation	Create the Electron media player and its playback/subtitle interface first, establishing the desktop workflow that later AI features extend.
5. Transcription and transcript context	Add Whisper to transcribe media into timed SRT cues. Parse cues into timestamp-and-text segments so later features can use the spoken content and link results to source moments.
6. Ask AI and local models	Connect relevant transcript excerpts to an Ollama text model for video questions. Configure local text/vision models and shape transcript context for longer learning tasks.
7. Learning tools and validation	Implement transcript-grounded flashcards with the text model and frame-plus-transcript storyboards with the vision model; validate structured responses, cache results, and test the workflows.

For flashcards, the service samples transcript chunks and expects structured cards tied to transcript cues and times. For storyboards, it selects scene candidates, extracts video frames, and pairs each frame with nearby transcript context before asking the vision model for an explanation. This extends the initial player and Ask AI pipeline with two different model inputs: text-only study cards and image-plus-text scene analysis.

System Architecture

The desktop application separates playback, orchestration, transcription, and local model inference. In the documented local workflow, media and transcript content stay on the user's computer.

Layer	Component	Responsibility
Interface	Electron renderer: index.html, renderer.js, styles.css	Loads media, handles playback/subtitles, sends actions, and renders results.
Application host	Electron main process: main.js	Creates the window, handles file selection and IPC, starts the backend, launches captions.
AI service	Express: server.js at 127.0.0.1:3000	Stores transcript context, answers questions, validates outputs, reports progress.
Speech/media	Python Whisper, PyTorch, FFmpeg	Transcribes speech, decodes media, and extracts storyboard frames.
Local inference	Ollama at 127.0.0.1:11434	Runs phi3 text generation and qwen2.5vl:3b vision generation.
Learning storage	Per-video .eduplay folder or home fallback	Stores analysis manifests and generated storyboard frames.

Workflow

1. Open a lecture file in the Electron player.

2. Generate Whisper captions or load an existing subtitle file.

3. Parse subtitle cues with timestamps and send them to the local service for the active media.

4. Use Ask AI, flashcards, or storyboard generation with the selected local model.

5. Validate the model output, cache finished learning artifacts, and display results in the interface.

Feature Workflows

Caption generation and transcript search

The caption generator loads Whisper's base model, selects CUDA when available and CPU otherwise, and transcribes the selected media. The renderer receives status updates and the resulting SRT path. Search reads active subtitle cues and seeks to their timestamps.

Caption transcription call

```
model = whisper.load_model(model_size, device=device)
result = model.transcribe(
    video_path,
    verbose=False,
    #task="translate",    Force English translation
    language=None      # Auto-detect input language
)
```

Source: generate_caption.py, lines 36–44

The translation task is commented out and language=None enables automatic detection. Despite the output suffix _AI_captions_en.srt, the current call does not force English translation.

Ask AI about this video

The subtitle manager parses SRT cues and sends them to /ask/init. For a question, the backend ranks transcript cues against the question and sends relevant excerpts to the text model. Model errors are shown in the interface rather than being hidden as empty answers.

Question-focused transcript selection

```
const relevantTranscript = selectQuestionContext(context?.segments || [], question, transcript);
```

Source: server.js, line 86

Flashcards

The Learning panel sends timestamped cues and a text model to /learning/flashcards. The backend samples chunks across long lectures, requests schema-constrained JSON, validates cue references, retries invalid output once, and saves cards in the per-video manifest. Cards support answer reveal, source review, timestamp jumps, and recall feedback.

Flashcard cache identity

```
const key = makeAnalysisKey(stat, transcriptKey, { textModel, artifact: "flashcards-v1" });
```

Source: server.js, line 105

Pictorial storyboard

The storyboard endpoint detects scene changes, spaces candidates across the duration, extracts resized frames, and sends each frame with nearby transcript context to qwen2.5vl:3b. It validates the response, streams progress, and caches the result. The sampler defaults to at most 24 frames and an 8-second minimum gap.

Representative scene selection

```
function selectRepresentativeTimestamps(candidates, duration, maxFrames = 24, minGap = 8) {
  const spaced = [];
  for (const time of candidates) {
    if (!spaced.length || time - spaced[spaced.length - 1] >= minGap) spaced.push(time);
  }
  if (spaced.length <= maxFrames) return spaced;
  const selected = [];
  const bins = Math.max(1, maxFrames);
```

Source: sceneDetection.js, lines 55–63

Data Handling and Privacy

- The local AI backend binds to 127.0.0.1:3000; Ollama uses 127.0.0.1:11434 by default.
- The documented workflow needs no cloud AI endpoint; selected media and transcripts are processed locally.
- Learning results are cached in a hidden per-video .eduplay directory, with a per-media home-folder fallback if needed.
- Cache identity includes media metadata, transcript fingerprint, selected model, and artifact version.
- The repo has no team dataset or lecture videos. Document data source and sharing permission before distribution.

Setup and Verification

For source development, install Node dependencies with npm ci, create and activate a Python 3.11 environment, install requirements.txt, install Ollama, pull phi3 and qwen2.5vl:3b, then run npm start. The app starts its local Express backend automatically.

The recorded local verification for this source baseline reported 10 passing unit tests and successful /ask, /learning/flashcards, and /learning/storyboard smoke requests using sample data. Rerun checks on the final build and capture the actual output before submission.

Evidence Placeholders

Replace each box with a legible screenshot from the actual running application. Use approved media and remove unrelated or personal desktop content.

Figure 1. Player and lecture

[INSERT ACTUAL SCREENSHOT OR EVIDENCE HERE]
Show video, playback controls, and visible subtitles.

Figure 2. Captions and transcript search

[INSERT ACTUAL SCREENSHOT OR EVIDENCE HERE]
Show captions and a search result with its timestamp.

Figure 3. Ask AI

[INSERT ACTUAL SCREENSHOT OR EVIDENCE HERE]
Show a transcript-grounded question and response.

Figure 4. Flashcards

[INSERT ACTUAL SCREENSHOT OR EVIDENCE HERE]
Show a card, revealed answer, source excerpt, and timestamp control.

Figure 5. Storyboard

[INSERT ACTUAL SCREENSHOT OR EVIDENCE HERE]
Show a frame, visual explanation, transcript context, and jump time.

Figure 6. Test evidence

[INSERT ACTUAL SCREENSHOT OR EVIDENCE HERE]
Insert the actual npm test output and test-machine details.

Research Paper and Evaluation

No research paper was present in the supplied repository. Insert the exact paper citation and only claims supported by that source. Do not imply the paper evaluated EduPlay unless the team performed that evaluation.

Paper detail	Team entry
Title and authors	[INSERT EXACT TITLE AND AUTHOR LIST]
Venue and year	[INSERT VENUE AND YEAR]
DOI or stable URL	[INSERT PAPER LINK]
Problem and method	[SUMMARIZE FROM THE PAPER]
Findings	[SUMMARIZE WITH PAGE OR SECTION REFERENCES]
Connection to EduPlay	[EXPLAIN WHAT THE PROJECT ADAPTS OR IMPLEMENTS]

Evaluation results: [INSERT ACTUAL TEST MEDIA, METHODS, METRICS, RESULTS, AND LIMITATIONS. Do not invent results.]

Limitations and Next Steps

- Caption quality depends on audio, language, model size, and available hardware.
- Local models can return incomplete or incorrect content; verify answers against transcript sources.
- Scene detection may miss gradual transitions or select non-instructional animation cuts.
- The repository does not include the paper, dataset, demo recording, or final screenshots.
- Run a complete demo on the target lecture and hardware before final submission.

Source repository: https://github.com/Drona-Srivastava/EduPlay